

פתרון שאלה 2, מועד 2019b-83

בשאלה 2 אנחנו צריכים להכניס לשפת PROC ביטוי חדש select, המקבל כפרמטרים רשימה של ביטויים וביטוי המהווה כפרדיקט. צריך להפעיל את הפרדיקט על כל אחד מהביטויים ברשימה ולהחזיר רשימה של אינדקסים שבהם הפרדיקט החזיר true.

איך פתרתי את שאלה 2?

- צריך לזכור שהשפות שלנו מחזירות תשובה מסוג expval, כאשר אנחנו מקבלים דרישה להחזיר תוצאה מסוג שאין לנו (num-val, bool-val, proc-val) אנחנו בעצם מתבקשים ליצור טיפוס חדש בשפה שלנו.
- כאשר אנחנו יוצרים טיפוס חדש אנחנו צריכים ליצור גם פרדיקט מתאים ומחלץ מתאים.

שלבי עבודה:

ראשית נעדכן את קובץ lang.scm ונוסיף את הדקדוק לשפה שלנו:

- את רשימת הביטויים מפרידים באמצעות רווחים ולכן נשתמש ב arbnos במקבל כמות שרירותית של ביטויים.

(expression

("select" "[" (arbno expression) "]" "where" expression)

select-exp)

לאחר מכן נעדכן את קובץ data-structures.scm ונוסיף טיפוס חדש לשפה:

- נכניס ל expval טיפוס חדש.

(define-datatype expval expval?

(num-val

(value number?))

(bool-val

(boolean boolean?))

(proc-val

(proc proc?))

(list-val

(lst lst?))) – our new type

- נגדיר פרדיקט מתאים, נשים לב כי אנחנו רוצים לתמוך ברשימה ריקה מכיוון שאין הכרח כי הביטוי חייב לקבל לפחות ביטוי אחד true.

```
(define lst?
```

```
(lambda (lst) ;;Checking if list of numbers only
```

```
(if (pair? lst)
```

```
(and (number? (car lst)) (if (null? (cdr lst)) #t (lst? (cdr lst))))
```

```
#t)));;This is true because our list can be empty!
```

- נגדיר מחליץ מתאים לטיפוס.

```
(define expval->lst
```

```
(lambda (v)
```

```
(cases expval v
```

```
(list-val (vlst) vlst)
```

```
(else ((expval-extractor-error 'num v))))))
```

סיימנו לעדכן את הקובץ data-structures.scm.

וכעת נעדכן את הקובץ interp.scm:

- נבצע בדיקה שהרשימה אינה ריקה, בהתאם לחוקיות השפה, ואז נעבור לפרצודרת עזר חיצונית.

```
(select-exp (exps pred)
```

```
(if (null? exps)
```

```
(eopl:error "You must give at least one exp to select!")
```

```
(let* ((newlst (list-val (list ))))
```

```
(apply-select exps pred env 1 newlst))))
```

- כעת נעבור לפרצדורת עזר חיצונית, פרצדורה זאת מגדירה תת-פרצדורה אנונימית המקבלת את רשימת הביטויים, ביטוי המהווה פרדיקט, סביבה נתונה בעת הפעלת הפרצדורה, אינדקס התחלתי (מוגדר ל-1) ורשימה ריקה התחלתית.
- נזכור כי מספור האינדקסים מתחיל מ-1, וכן הרשימה יכולה להיות ריקה.

```
(define apply-select
```

```
  (lambda (exps pred env index lst)
```

```
    (if (null? exps)
```

```
        lst
```

```
        (let* ((e1 (value-of (call-exp pred (car exps)) env))
```

```
                (newlst (list-val (append (expval->lst lst) (list index))))))
```

```
        (if (expval->bool e1)
```

```
            (apply-select (cdr exps) pred env (+ index 1) newlst)
```

```
            (apply-select (cdr exps) pred env (+ index 1) lst))))))
```